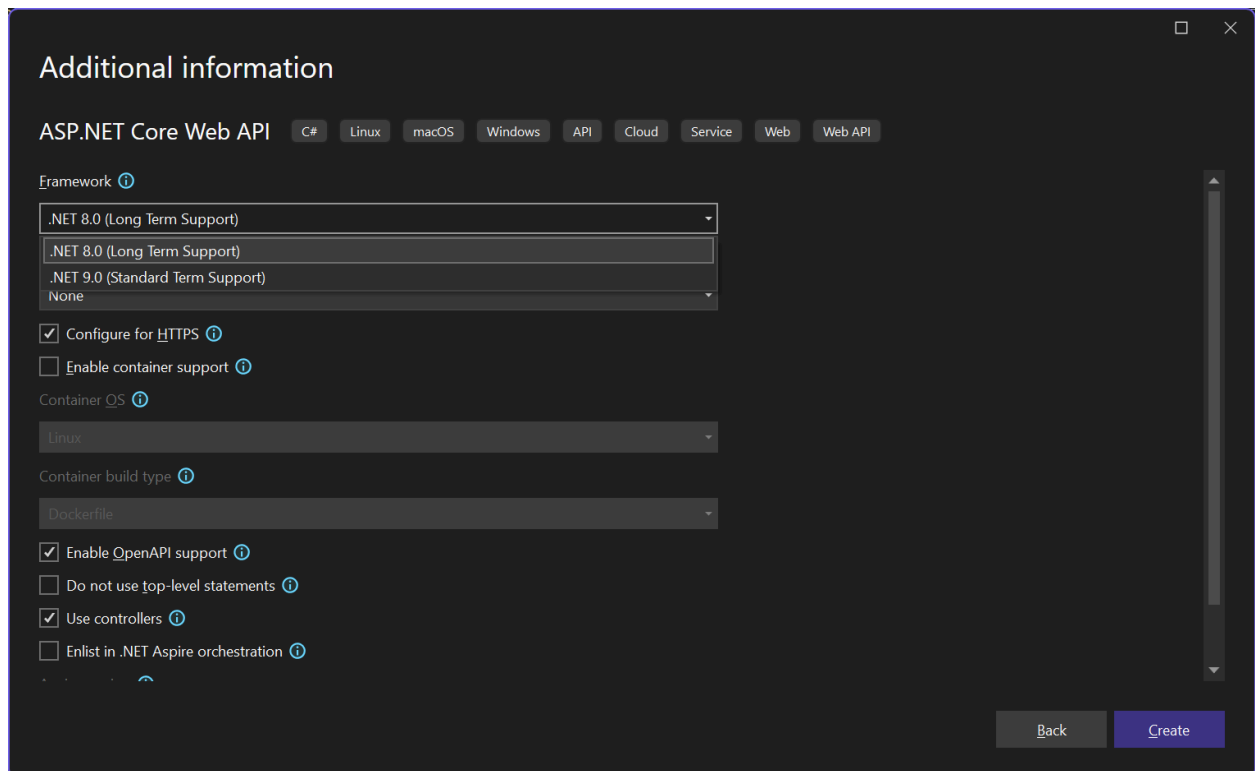
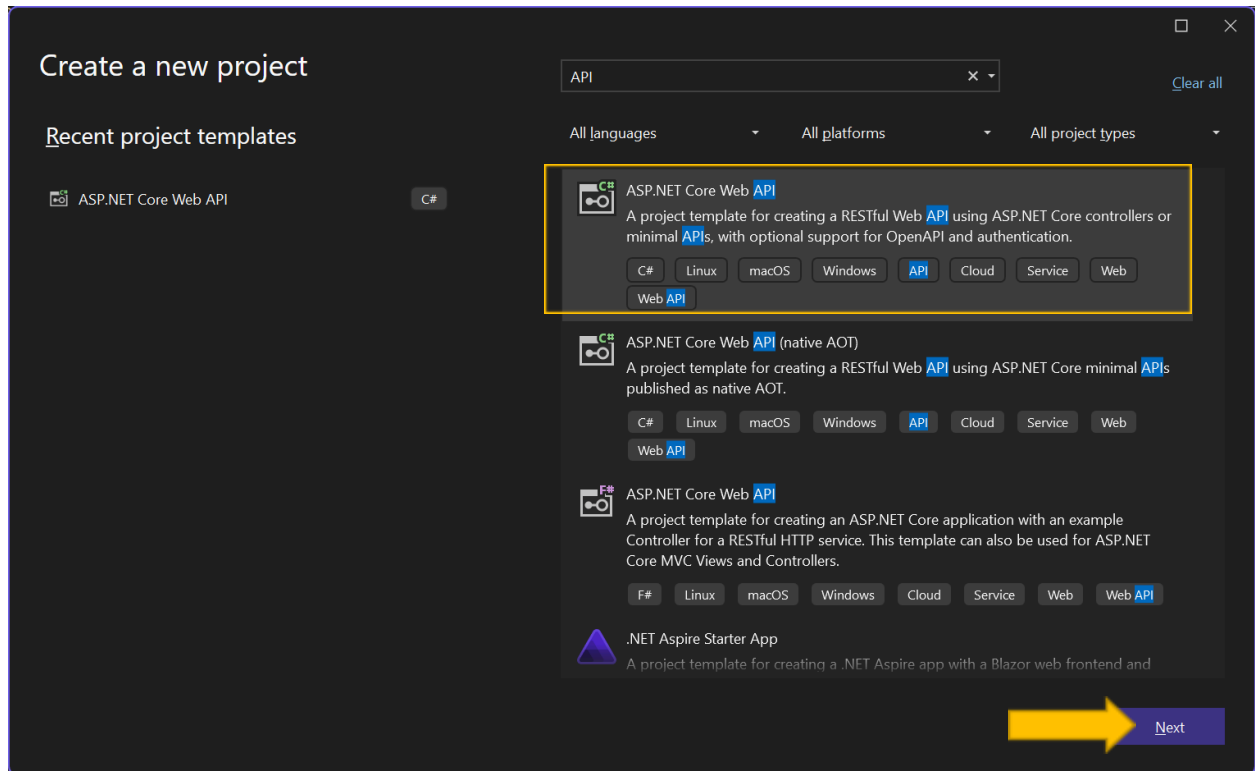
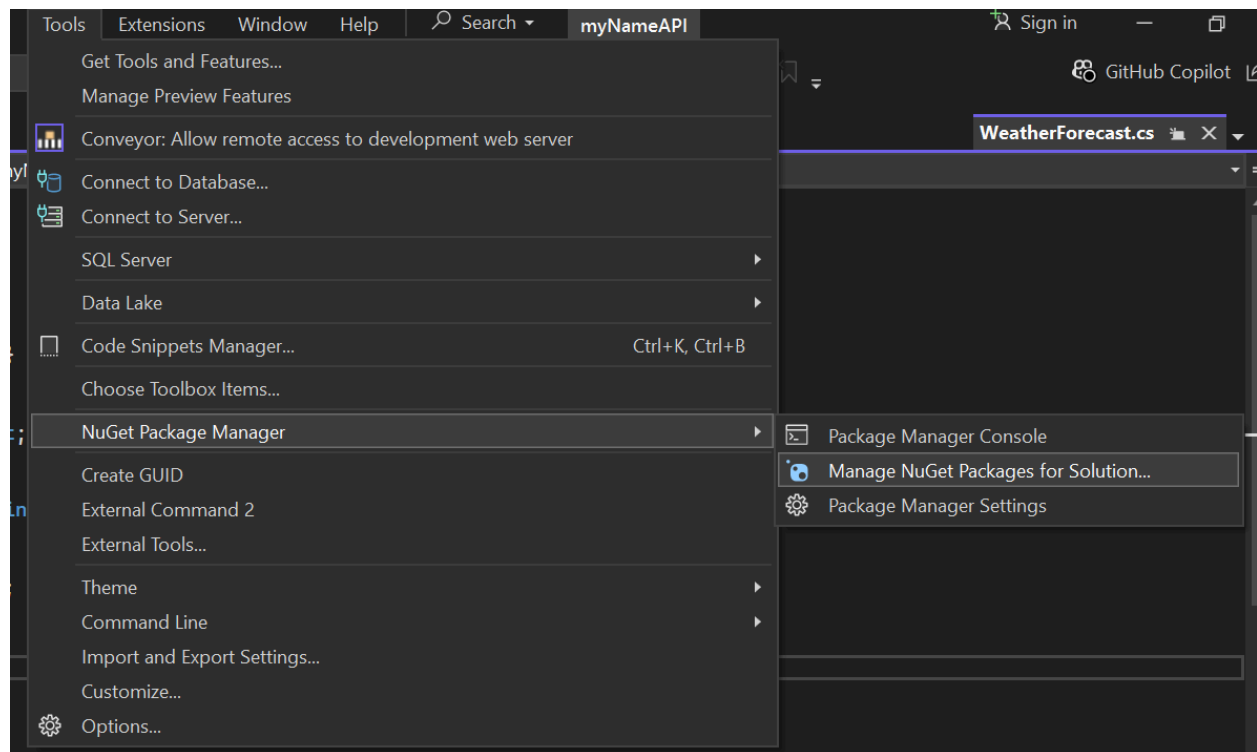




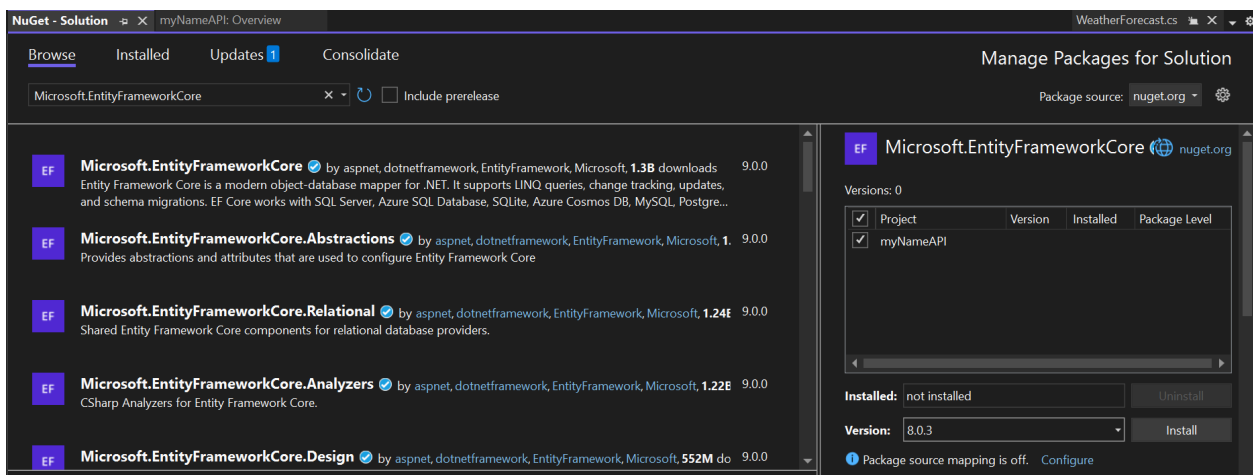
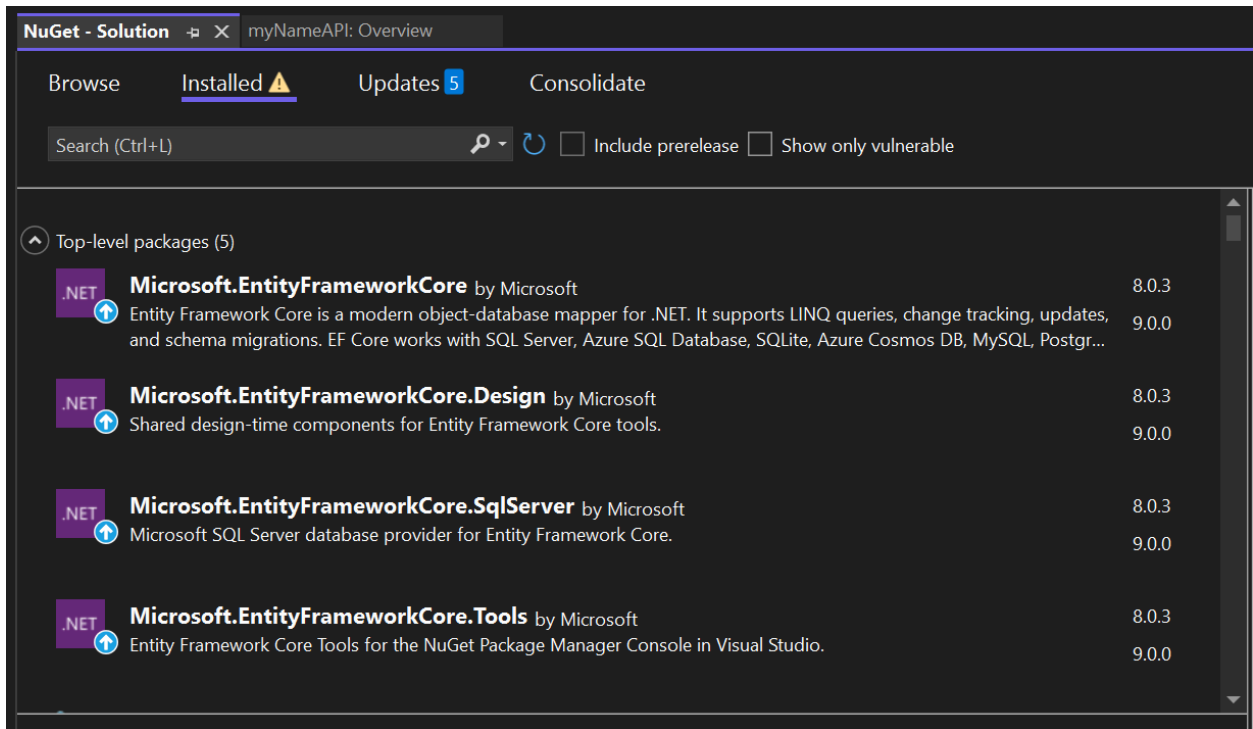
Cài Đặt Gói NuGet Cần Thiết

Cài đặt các gói sau bằng NuGet:

- **Microsoft.EntityFrameworkCore** phiên bản 8.0.3
- **Microsoft.EntityFrameworkCore.Design** phiên bản 8.0.3
- **Microsoft.EntityFrameworkCore.SqlServer** phiên bản 8.0.3 (hoặc provider cơ sở dữ liệu khác)
- **Microsoft.EntityFrameworkCore.Tools** phiên bản 8.0.3



Kiểm tra các gói đã cài đặt



Dưới đây là một ví dụ đơn giản về cách xây dựng một RESTful API cho Product trong ASP.NET Core, sử dụng Controller và Repository Pattern.

Tạo thư mục models trong Project

Tạo Class Product trong thư mục Models

```
public class Product
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public decimal Price { get; set; }
    public string? Image { get; set; }
    public string? Description { get; set; }
    // Other properties
}
```

Tạo Class **IProductRepository** trong thư mục Repositories

```
public interface IProductRepository
{
    Task<IEnumerable<Product>> GetProductsAsync();
    Task<Product> GetProductByIdAsync(int id);
    Task AddProductAsync(Product product);
    Task UpdateProductAsync(Product product);
    Task DeleteProductAsync(int id);
}
```

Tạo Class **ProductRepository** trong thư mục Repositories

```
public class ProductRepository : IProductRepository
{
    private readonly ApplicationDbContext _context;
    public ProductRepository(ApplicationDbContext context)
    {
        _context = context;
    }
    public async Task<IEnumerable<Product>> GetProductsAsync()
    {
        return await _context.Products.ToListAsync();
    }
    public async Task<Product> GetProductByIdAsync(int id)
    {
        return await _context.Products.FindAsync(id);
    }
    public async Task AddProductAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
    }
    public async Task UpdateProductAsync(Product product)
    {
        _context.Entry(product).State = EntityState.Modified;
        await _context.SaveChangesAsync();
    }
    public async Task DeleteProductAsync(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product != null)
        {
            _context.Products.Remove(product);
        }
    }
}
```

```

        await _context.SaveChangesAsync();
    }
}

```

Tạo Class `ApplicationDbContext` trong thư mục Models và cấu hình

```

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }
    public DbSet<Product> Products { get; set; }
}

```

Cấu hình EF Core trong file `Program.cs`:

```

var builder = WebApplication.CreateBuilder(args);
//Cấu hình: Dùng SQL Server làm hệ quản trị csdl cho ứng dụng
builder.Services.AddDbContext<ApplicationDbContext>(options
=>options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"
)));
//Cấu hình khác...

```

Cấu hình connection string trong `appsettings.json`.

```

{
    //Chỉ cho chương trình biết tên server và tên DB muốn đặt
    "ConnectionStrings": {
        "DefaultConnection":
"Server=MY-PC\\MRHUNG189;Database=myNameAPI;Trusted_Connection=True;TrustServerCertificate=True"
    },
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*"
}

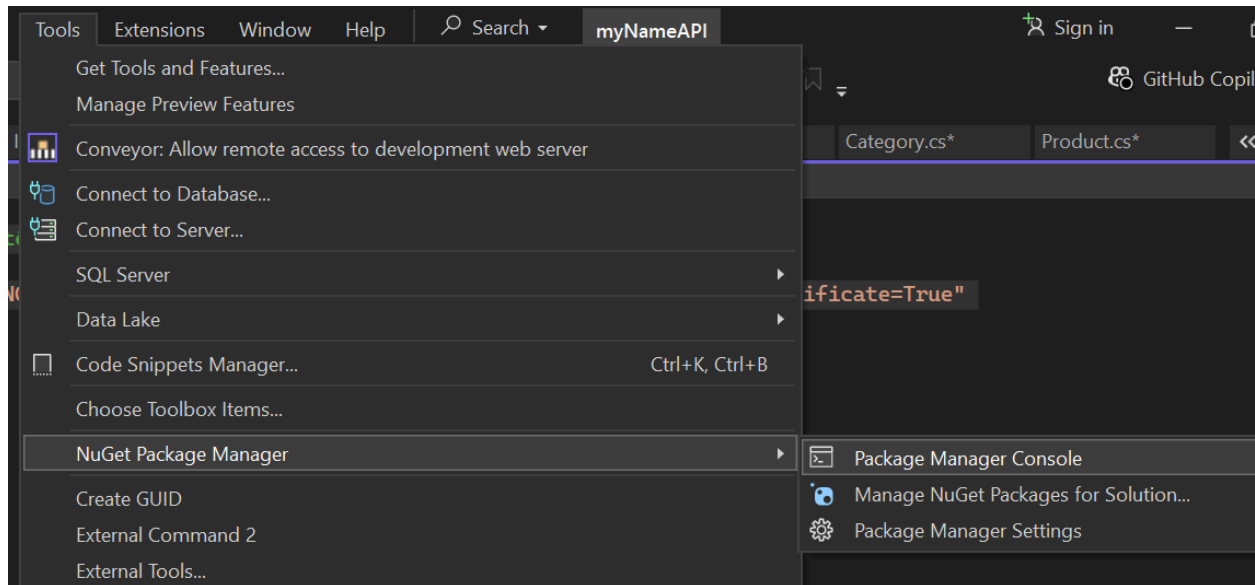
```

Sử dụng migrations để ánh xạ và cập nhật các bảng vào trong CSDL

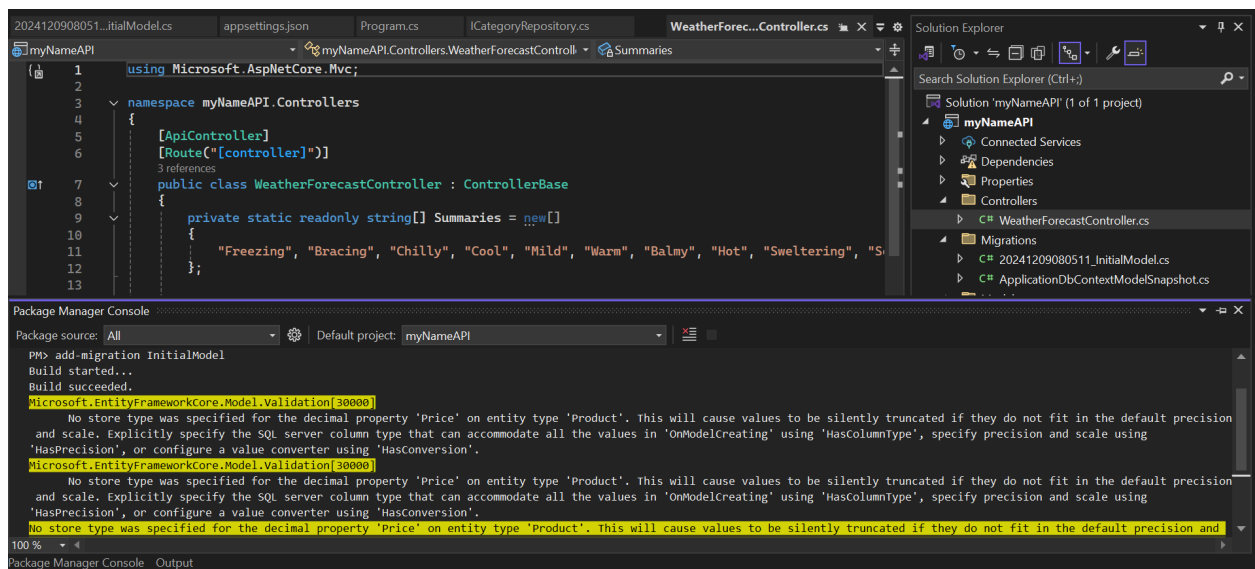
Entity Framework Code First

Add-Migration Initial

Update-Database



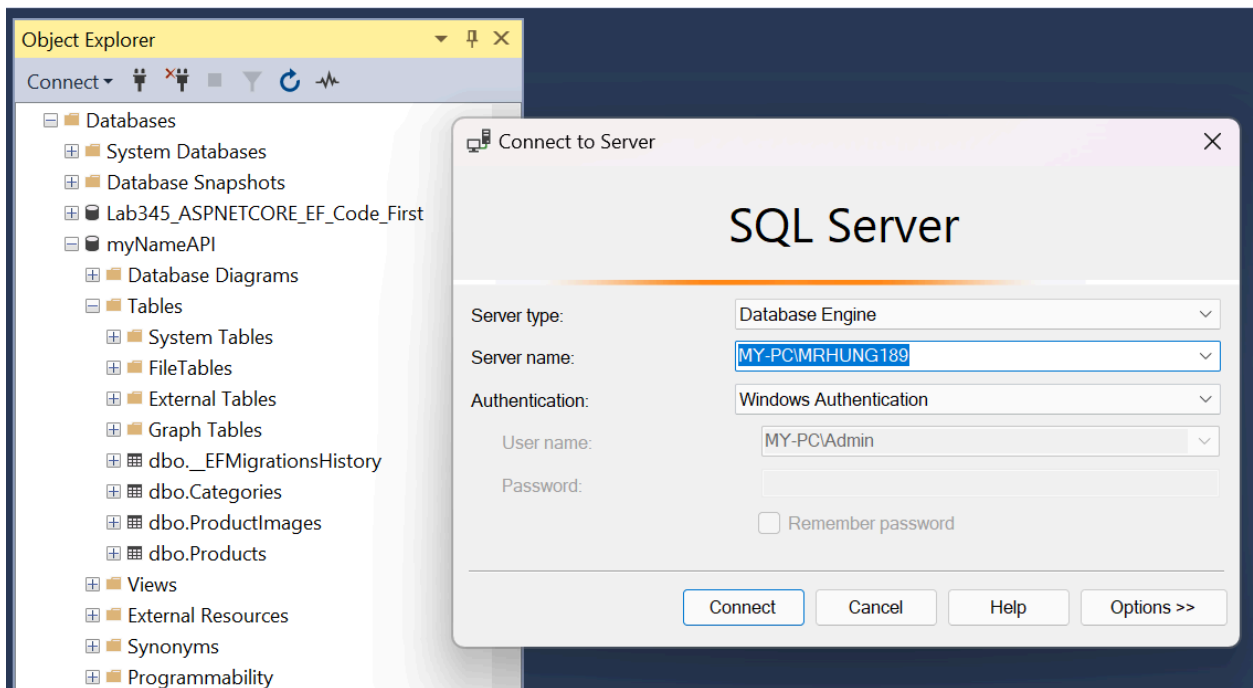
Lệnh: add-migration InitialModel



Lệnh: update-database

```
PM> update-database
Build started...
Build succeeded.
Microsoft.EntityFrameworkCore.Model.Validation[30000]
    No store type was specified for the decimal property 'Price' on entity type 'Product'. This will cause values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType', specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
Microsoft.EntityFrameworkCore.Model.Validation[30000]
    No store type was specified for the decimal property 'Price' on entity type 'Product'. This will cause values to be silently truncated if they do not fit in the default precision and scale. Explicitly specify the SQL server column type that can accommodate all the values in 'OnModelCreating' using 'HasColumnType', specify precision and scale using 'HasPrecision', or configure a value converter using 'HasConversion'.
Microsoft.EntityFrameworkCore.Database.Command[20101]
    Executed DbCommand (197ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
    CREATE DATABASE [myNameAPI];
Microsoft.EntityFrameworkCore.Database.Command[20101]
    Executed DbCommand (96ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
    IF SERVERPROPERTY('EngineEdition') <> 5
    BEGIN
```

Kiểm tra CSDL đã được cập nhật trong SQL Server chưa?



Tạo Class **ProductApiController** trong thư mục Controllers

```
[Route("api/[controller]")]
[ApiController]

public class ProductApiController : ControllerBase
{
    private readonly IProductRepository _productRepository;
    public ProductApiController(IProductRepository productRepository)
    {
        _productRepository = productRepository;
    }
    [HttpGet]
    public async Task<IActionResult> GetProducts()
    {
        try
```

```

    {
        var products = await _productRepository.GetProductsAsync();
        return Ok(products);
    }
    catch (Exception ex)
    {
        // Handle exception
        return StatusCode(500, "Internal server error");
    }
}

[HttpGet("{id}")]
public async Task<IActionResult> GetProductById(int id)
{
    try
    {
        var product = await _productRepository.GetProductByIdAsync(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }
    catch (Exception ex)
    {
        // Handle exception
        return StatusCode(500, "Internal server error");
    }
}

[HttpPost]
public async Task<IActionResult> AddProduct([FromBody] Product product)
{
    try
    {
        await _productRepository.AddProductAsync(product);
        return CreatedAtAction(nameof(GetProductById), new
        {
            id =
            product.Id
        }, product);
    }
    catch (Exception ex)
    {
        // Handle exception
        return StatusCode(500, "Internal server error");
    }
}

[HttpPut("{id}")]
public async Task<IActionResult> UpdateProduct(int id, [FromBody] Product
product)
{
    try
    {
        if (id != product.Id)
            return BadRequest();
        await _productRepository.UpdateProductAsync(product);
        return NoContent();
    }
    catch (Exception ex)

```

```

        {
            // Handle exception
            return StatusCode(500, "Internal server error");
        }
    }
    [
        HttpDelete("{id}")]
        public async Task<IActionResult> DeleteProduct(int id)
        {
            try
            {
                await _productRepository.DeleteProductAsync(id);
                return NoContent();
            }
            catch (Exception ex)
            {
                // Handle exception
                return StatusCode(500, "Internal server error");
            }
        }
    }
}

```

Mở file **Program.cs** và bổ sung cấu hình

```

using Microsoft.EntityFrameworkCore;
using testAPI.models;
using testAPI.Repositories;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

// Add services to the container.

builder.Services.AddControllers();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
// Configure CORS: Cross-Origin Resource Sharing, được dịch là "Chia sẻ tài nguyên
giữa các nguồn gốc khác nhau
builder.Services.AddCors(options =>
{
    options.AddPolicy(name: "MyAllowOrigins", policy =>
    {
        //Thay bằng địa chỉ localhost khi khởi chạy bên frontend (VSCode)
        policy.WithOrigins("http://127.0.0.1:5500", "http://localhost:5500")
        .AllowAnyHeader()
        .AllowAnyMethod();
    });
});

var app = builder.Build();

```

```
// Middleware for production error handling
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/error"); // Custom error handling endpoint
    app.UseHsts(); // Enforce HTTPS in production
}
else
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

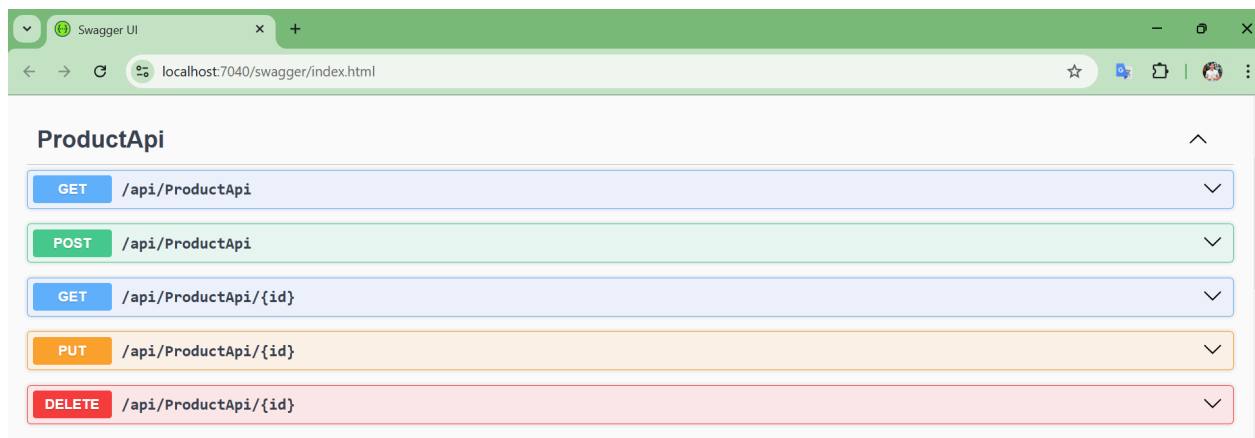
app.UseHttpsRedirection();
// Áp dụng CORS cho các yêu cầu đến API
app.UseCors("MyAllowOrigins");

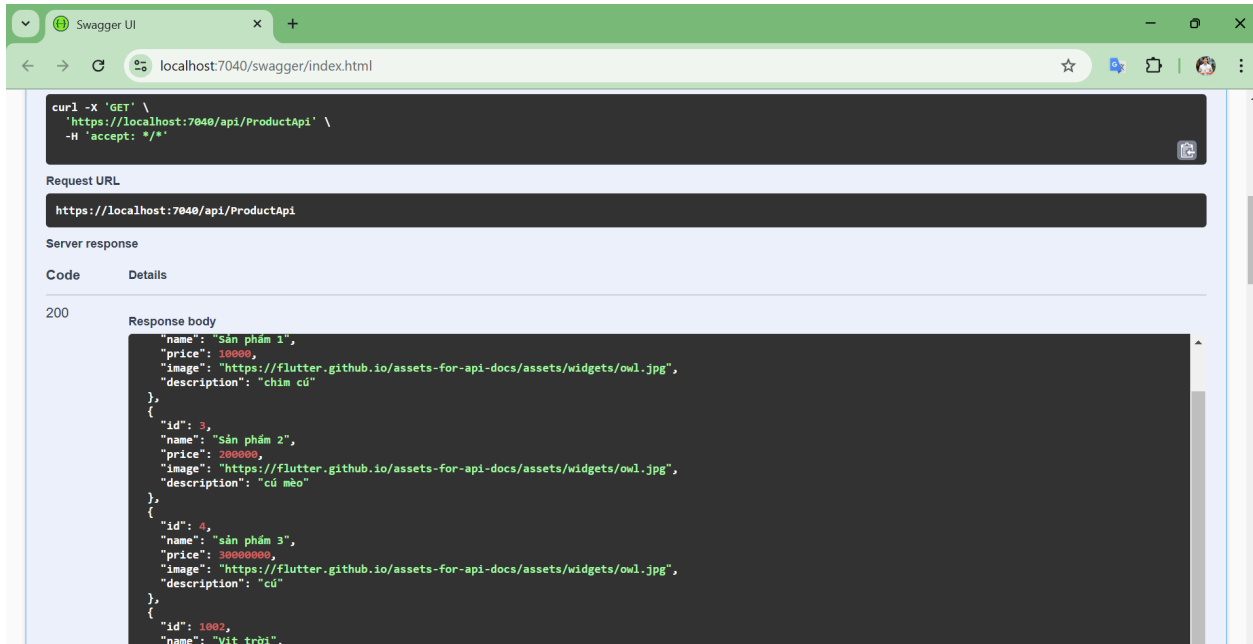
app.UseAuthorization();

app.MapControllers();

app.Run();
```

chạy thử chương trình và xem kết quả API (nhớ thêm dữ liệu vào DB trước)





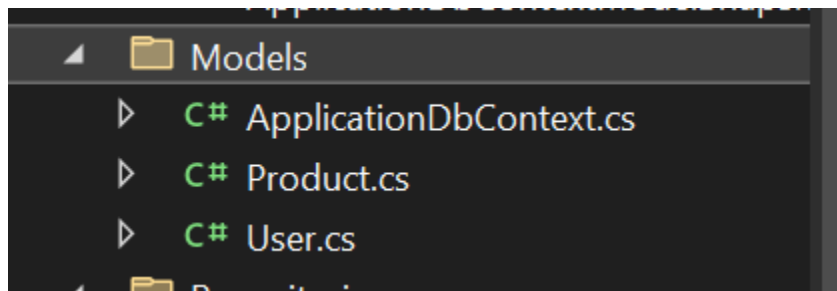
Json nhận được từ API trả về

```
{
  "id": 2,
  "name": "Sản phẩm 1",
  "price": 10000,
  "image": "https://flutter.github.io/assets-for-api-docs/assets/widgets/owl.jpg",
  "description": "chim cú"
},
{
  "id": 3,
  "name": "Sản phẩm 2",
  "price": 200000,
  "image": "https://flutter.github.io/assets-for-api-docs/assets/widgets/owl.jpg",
  "description": "cú mèo"
},
{
  "id": 4,
  "name": "sản phẩm 3",
  "price": 30000000,
  "image": "https://flutter.github.io/assets-for-api-docs/assets/widgets/owl.jpg",
  "description": "cú"
},
{
  "id": 1002,
  "name": "Vịt trời",
  "price": 10000,
  "image": "Cập nhật sau",
  "description": "Con này làm món gì ta?"
}
```

Có thể dùng Live Server trên Visual Code để chạy server hoặc dùng ứng dụng bên thứ 3 (Conveyor)

Tiếp tục bài tuần trước

Bước 1. Tạo class User trong thư mục Model



```
public class User: IdentityUser
{
    public string? Initials { get; set; }
    //User = IdentityUser + string Initials
}
```

Bước 2: Cập nhật ApplicationDbContext

```
public class ApplicationDbContext : IdentityDbContext<User>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options) : base(options) { }
    public DbSet<Product> Products { get; set; }
    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        builder.Entity<User>().Property(u=>u.Initials).HasMaxLength(5);
        builder.HasDefaultSchema("identity");
    }
}
```

Bước 3. Cài đặt thêm các gói cần thiết

‘Microsoft.AspNetCore.Identity.EntityFrameworkCore’ Phiên bản 8.0.3. Đây là gói giúp thực hiện các thao tác: đăng ký, đăng nhập, cập nhật thông tin User, phân quyền, mã hóa mật khẩu....

‘Microsoft.AspNetCore.Authentication.JwtBearer’ Phiên bản 8.0.3. Đây là gói giúp thực tạo các middleware, token chứa các thông tin về tài khoản, quyền...

Bước 4. Cấu hình file Program.cs

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Sang6_WebAPI.Models;
using Sang6_WebAPI.Repositories;
using System.Text;

var builder = WebApplication.CreateBuilder(args);
//Cấu hình: Dùng SQL Server làm hệ quản trị csdl cho ứng dụng
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

// Add services to the container.

builder.Services.AddControllers();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();


// Configure CORS: Cross-Origin Resource Sharing, được dịch là "Chia sẻ tài nguyên
giữa các nguồn gốc khác nhau
builder.Services.AddCors(options =>
{
    options.AddPolicy(name: "MyAllowOrigins", policy =>
    {
        //Thay bằng địa chỉ localhost khi khởi chạy bên frontend (VSCode)
        policy.WithOrigins("http://127.0.0.1:5500", "http://localhost:5500")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

builder.Services.AddAuthorization();

var app = builder.Build();

// Middleware for production error handling
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/error"); // Custom error handling endpoint
    app.UseHsts(); // Enforce HTTPS in production
}
else
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

```

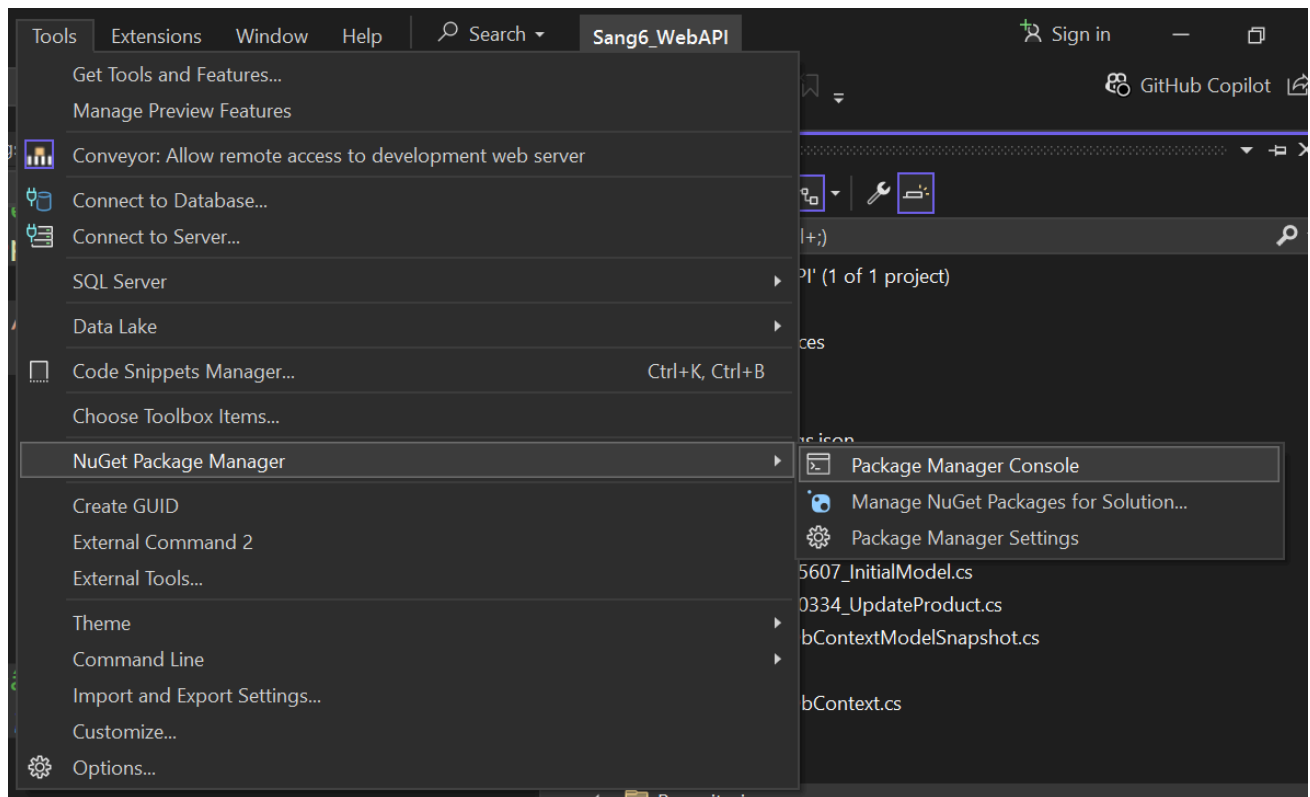
```
// Áp dụng CORS cho các yêu cầu đến API
app.UseCors("MyAllowOrigins");
app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapIdentityApi<User>();
```

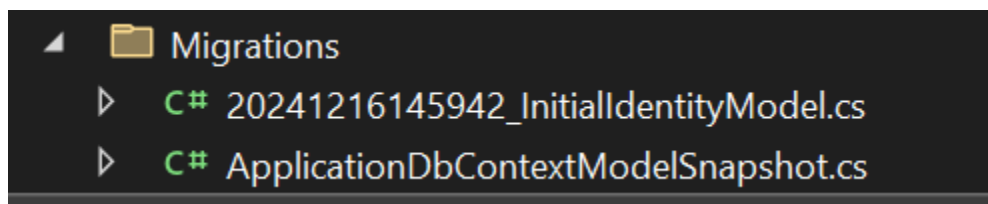
```
app.MapControllers();
```

```
app.Run();
```

Bước 5. Thực hiện add-migration và update-database

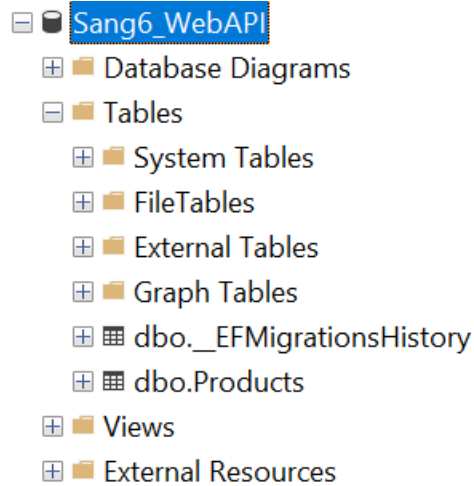


Sau khi **add-migration InitialIdentityModel**

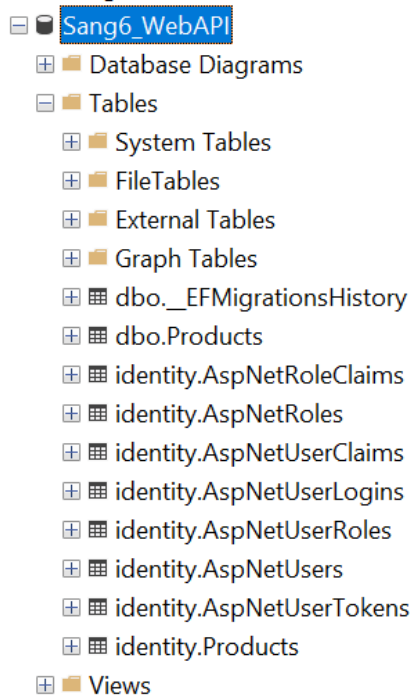


Thực hiện lệnh **update-database** để ánh xạ Identity vào cơ sở dữ liệu

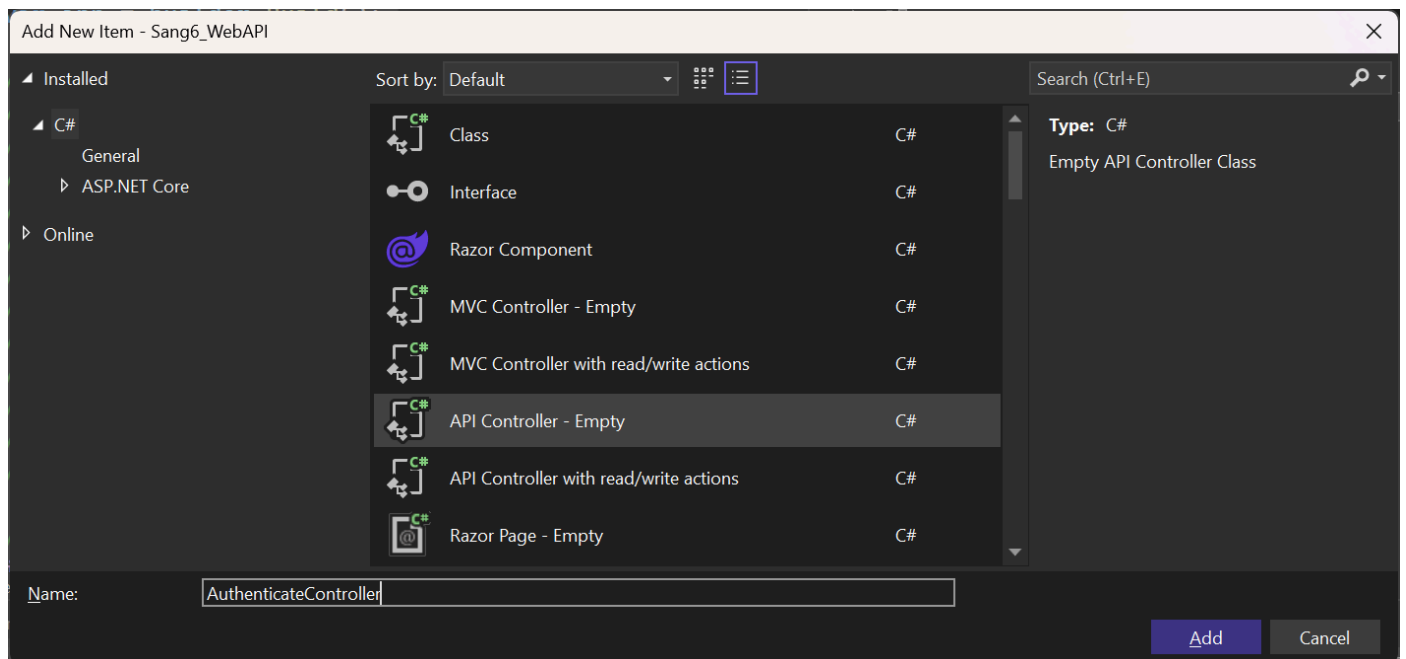
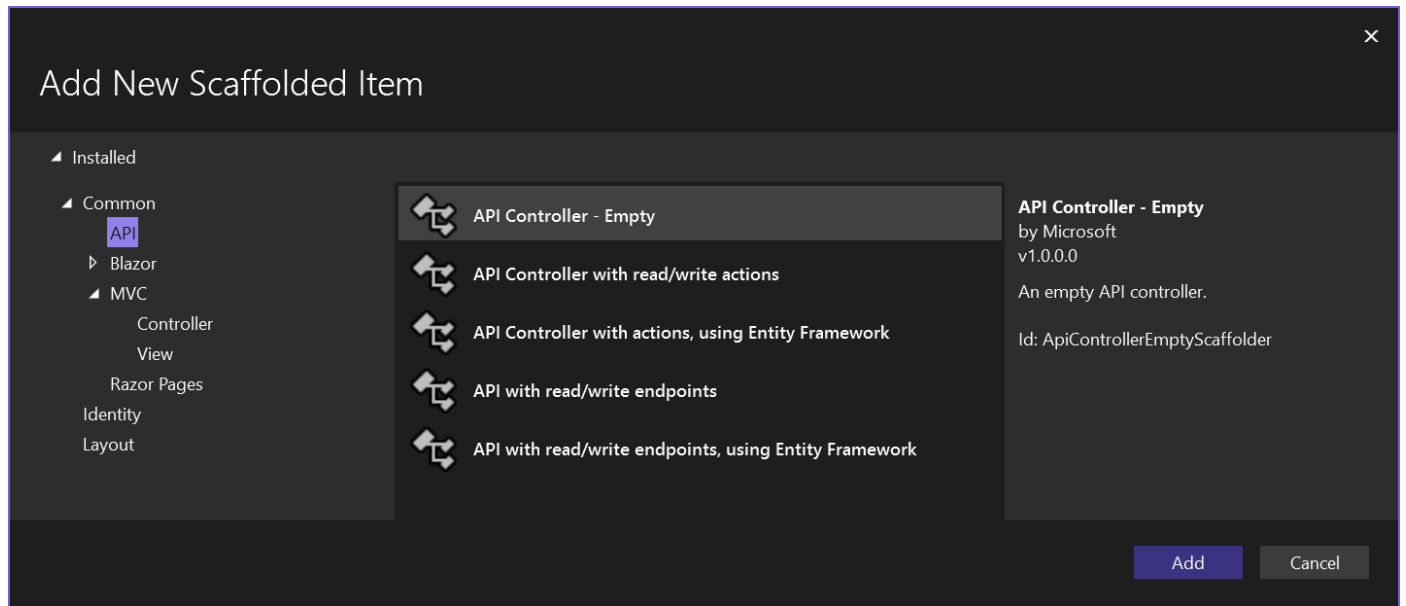
Trước khi ánh xạ



Sau khi ánh xạ



Bước 6. Trong thư mục controller, thêm ApiController tên **AuthenticateController**



```
public class AuthenticateController : ControllerBase
{
    private readonly UserManager<User> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;
    private readonly IConfiguration _configuration;

    public AuthenticateController(
        UserManager<User> userManager,
        RoleManager<IdentityRole> roleManager,
```

```

    IConfiguration configuration)
    {
        _userManager = userManager;
        _roleManager = roleManager;
        _configuration = configuration;
    }

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegistrationModel model)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);

        var userExists = await _userManager.FindByNameAsync(model.Username);
        if (userExists != null)
            return StatusCode(StatusCodes.Status400BadRequest, new { Status = false,
Message = "User already exists" });

        var user = new User
        {
            UserName = model.Username,
            Email = model.Email,
            Initials = model.Initials
        };

        var result = await _userManager.CreateAsync(user, model.Password);
        if (!result.Succeeded)
            return StatusCode(StatusCodes.Status500InternalServerError, new { Status
= false, Message = "User creation failed" });

        // Assign Role if Provided
        if (!string.IsNullOrEmpty(model.Role))
        {
            if (!await _roleManager.RoleExistsAsync(model.Role))
            {
                await _roleManager.CreateAsync(new IdentityRole(model.Role));
            }
            await _userManager.AddToRoleAsync(user, model.Role);
        }

        return Ok(new { Status = true, Message = "User created successfully" });
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginModel model)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);

        var user = await _userManager.FindByNameAsync(model.Username);
        if (user == null || !await _userManager.CheckPasswordAsync(user,
model.Password))
            return Unauthorized(new { Status = false, Message = "Invalid username or
password" });

        var userRoles = await _userManager.GetRolesAsync(user);

        var authClaims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, user.UserName),

```

```

        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    foreach (var userRole in userRoles)
    {
        authClaims.Add(new Claim(ClaimTypes.Role, userRole));
    }

    var token = GenerateToken(authClaims);
    return Ok(new { Status = true, Message = "Logged in successfully", Token = token });
}

private string GenerateToken(IEnumerable<Claim> claims)
{
    var jwtSettings = _configuration.GetSection("JWTKey");
    var authSigningKey = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["Secret"]));

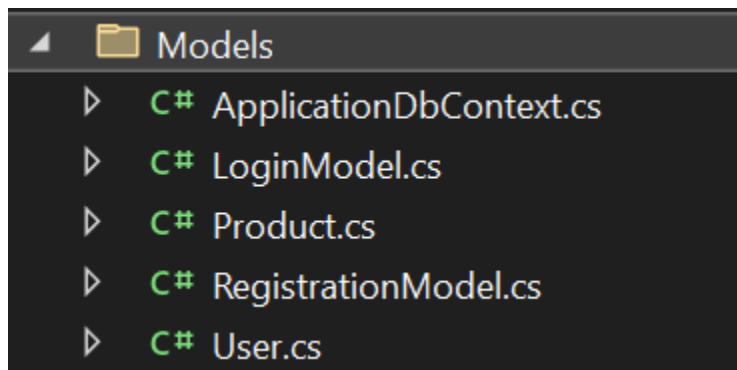
    var tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(claims),
        Expires =
    DateTime.UtcNow.AddHours(Convert.ToDouble(jwtSettings["TokenExpiryTimeInHour"])),
        Issuer = jwtSettings["ValidIssuer"],
        Audience = jwtSettings["ValidAudience"],
        SigningCredentials = new SigningCredentials(authSigningKey,
    SecurityAlgorithms.HmacSha256)
    };

    var tokenHandler = new JwtSecurityTokenHandler();
    var token = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(token);
}
}

```

Bước 7. Thêm các Class cần thiết

Chương trình đang báo lỗi tại `LoginModel` và `RegistrationModel`, trong thư mục Model thêm hai class có tên trên



```

public class LoginModel
{

```

```

    [Required]
    public string Username { get; set; } = string.Empty;
    [Required]
    public string Password { get; set; } = string.Empty;
}

public class RegistrationModel
{
    [Required]
    public string Username { get; set; } = string.Empty;
    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;
    [Required, MinLength(6)]
    public string Password { get; set; } = string.Empty;
    public string? Initials { get; set; }
    public string? Role { get; set; } // Optional - assign a role if needed
}

```

Bước 8. Cập nhật Program.cs (Nhớ kiểm tra đã cài đặt gói **JwtBearer** chưa ???)

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Sang6_WebAPI.Models;
using Sang6_WebAPI.Repositories;
using System.Text;

var builder = WebApplication.CreateBuilder(args);
//Cấu hình: Dùng SQL Server làm hệ quản trị csdl cho ứng dụng
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")))
);

// Register identity
builder.Services.AddIdentity<User, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

// Add services to the container.

builder.Services.AddControllers();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure CORS: Cross-Origin Resource Sharing, được dịch là "Chia sẻ tài nguyên
giữa các nguồn gốc khác nhau

```

```

builder.Services.AddCors(options =>
{
    options.AddPolicy(name: "MyAllowOrigins", policy =>
    {
        //Thay bằng địa chỉ localhost khi khởi chạy bên frontend (VSCode)
        policy.WithOrigins("http://127.0.0.1:5500", "http://localhost:5500")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

//Configure JWT Authentication
var jwtSettings = builder.Configuration.GetSection("JWTKey");
var key = Encoding.UTF8.GetBytes(jwtSettings["Secret"]);

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = "Bearer";
    options.DefaultChallengeScheme = "Bearer";
})
.AddJwtBearer("Bearer", options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["ValidIssuer"],
        ValidAudience = jwtSettings["ValidAudience"],
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };
});

builder.Services.AddAuthorization();

var app = builder.Build();

//Tạo ra các Role trong ứng dụng để sau này thực hiện phân quyền
using (var scope = app.Services.CreateScope())
{
    var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    var roles = new[] { "Admin", "User" };
    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
        {
            await roleManager.CreateAsync(new IdentityRole(role));
        }
    }
}

// Middleware for production error handling
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/error"); // Custom error handling endpoint
    app.UseHsts(); // Enforce HTTPS in production
}

```

```

}
else
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
// Áp dụng CORS cho các yêu cầu đến API
app.UseCors("MyAllowOrigins");
app.UseAuthentication();
app.UseAuthorization();

//Người dùng đã tự config Identity user nên bỏ phần này
//app.MapIdentityApi<User>();

app.MapControllers();

app.Run();

```

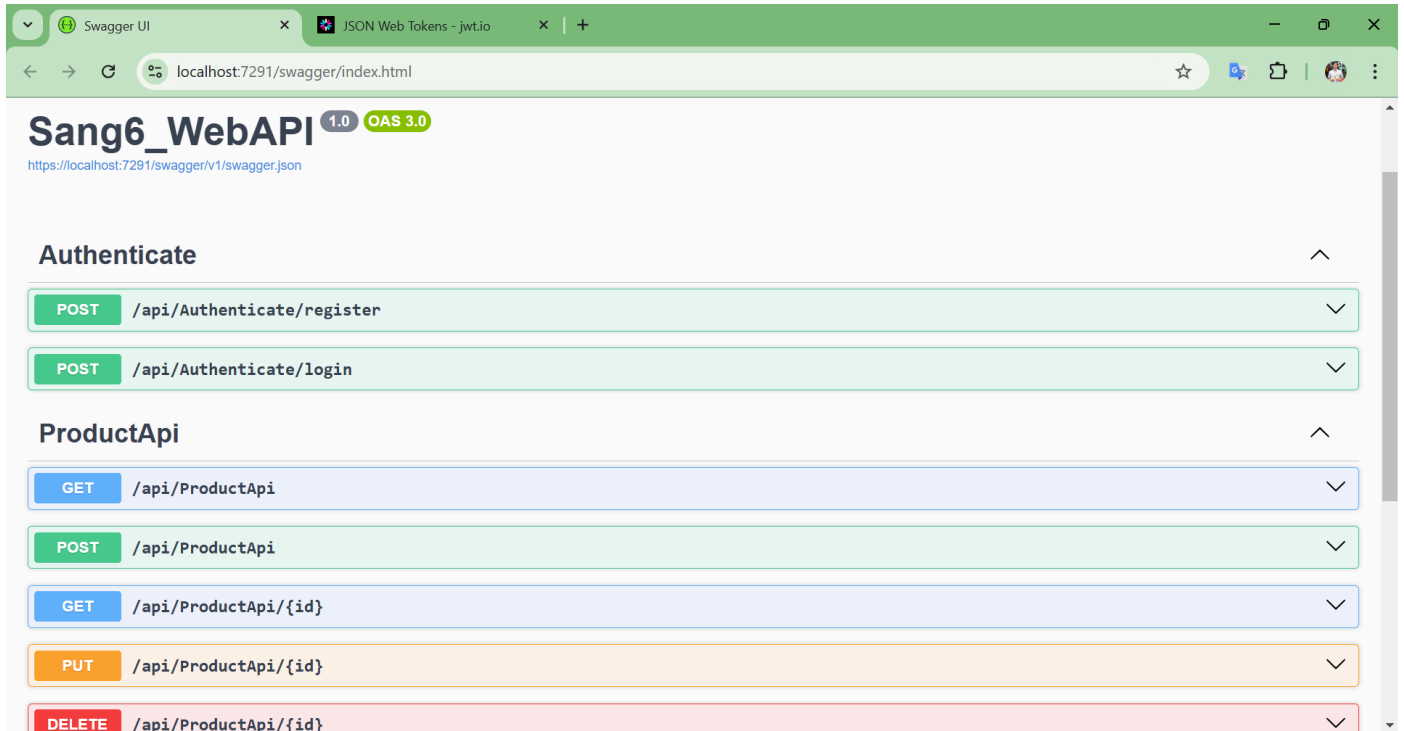
Bước 9. Cập nhật lại file appsetting.json

```

{
    //Cấu hình thông tin khóa mã hóa JWT
    "JWTKey": {
        "Secret": "AReallyLongSecretKeyForYourJWTTokenSigning!", //Khóa bí mật để khóa
        và giải khóa token
        "ValidIssuer": "YourIssuer", //Thông tin người phát hành
        "ValidAudience": "YourAudience", //Ứng dụng nhận được token này
        "TokenExpiryTimeInHour": "1" //Thời gian hiệu lực của token
    },
    "ConnectionStrings": {
        "DefaultConnection":
"Server=MY-PC\\MRHUNG189;Database=testAPI;Trusted_Connection=True;TrustServerCertificate=True"
    },
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*"
}

```

Bước 10. Debug chương trình và đăng ký tài khoản, đăng nhập



Tiến hành đăng ký và đăng nhập thử

```
{  
  "username": "manhhung3",  
  "email": "manhhung3@gmail.com",  
  "password": "Manhhung@123",  
  "initials": "hung3",  
  "role": "User"  
}
```

Đăng ký thành công

```
curl -X 'POST' \
  'https://localhost:7291/api/Authenticate/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "manhhung3",
    "email": "manhhung3@gmail.com",
    "password": "Manhhung@123",
    "initials": "hung3",
    "role": "User"
  }'
```

Request URL

```
https://localhost:7291/api/Authenticate/register
```

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "status": true,
  "message": "User created successfully"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 16 Dec 2024 15:41:41 GMT
server: Kestrel
```

Responses

Đăng nhập thành công

